

C++ Coroutine TS Issues

Doc. no. P0664R0
Date: Revised 2017-06-18 at 14:25:00 UTC
Project: Programming Language C++
Reference: ISO/IEC PDIS 22277, C++ Extensions for Coroutines
Audience: EWG, CWG, LWG
Reply to: Gor Nishanov <gorn@microsoft.com>

Introduction

All proposed resolutions wording is relative to [N4663](#) (ISO/IEC PDIS 22277).

Table of content

- Core issues with proposed wording: [2](#) [6](#) [7](#) [8](#) [18](#) [21](#)
- Core comments requesting rebase of a TS to C++17: [3](#) [5](#) [17](#)
- Core comments (no action): [4](#)
- LWG issues with proposed wording: [9](#) [10](#) [22](#) [23](#)
- LWG issues with no wording: [11](#) [12](#) [14](#) [15](#) [16](#) [19](#) [20](#)
- Evolution issues with no wording: [1](#) [13](#)

Core Issues (with proposed wording)

[2.](#) Change to italics *await-resume* in 5.3.8/4

Section: 5.3.8 [expr.await] **Status:** Has wording **Submitter:** US002 **Opened:** 2017-06-05 **Last modified:** 2017-06-05

Proposed resolution:

Modify 5.3.8/4

The *await-expression* has the same type and value category as the ~~*await-resume*~~*await-resume* expression.

[6.](#) Remove or update stateful allocator example in 8.4.4/12

Section: 8.4.4 [dcl.fct.def.coroutine] **Status:** Has wording **Submitter:** US006 **Opened:** 2017-06-05 **Last modified:** 2017-06-05

Issue:

Stateful allocators (pmr) do not work this way, there's no mechanism for allocator propagation to the captured state.

Strike section 12, or provide mechanism for holding allocator.

Proposed resolution:

Remove example 8.4.4/12

[7.](#) Fix generator example in 8.4.4/11

Section: 8.4.4/11 [dcl.fct.def.coroutine] **Status:** Has wording **Submitter:** US007 **Opened:** 2017-06-05 **Last modified:** 2017-06-05

Issue:

Is `unhandled_exception()` a requirement for a `promise_type`?

a) Call `std::terminate` if not present

or

b) Add `unhandled_exception()` to the complete example of `promise_type` in 8.4.4 paragraph 11, the generator example.

Discussion:

`unhandled_exception()` is required to be present in a `promise_type`. There are more mistakes in the example that are fixed in the proposed resolution. Also, required includes are added to make the example self contained and runnable in [online compilers](#).

Proposed resolution:

Modify the example in 8.4.4/11 as follows:

```
#include <iostream>
#include <experimental/coroutine>

// ::operator new(size_t, nothrow_t) will be used if allocation is needed
struct generator {
    struct promise_type;
    using handle = std::experimental::coroutine_handle<promise_type>;
    struct promise_type {
        int current_value;
        static auto get_return_object_on_allocation_failure() { return generator{nullptr}; }
        auto get_return_object() { return generator{handle::from_promise(*this)}; }
        auto initial_suspend() { return std::experimental::suspend_always{}; }
        auto final_suspend() { return std::experimental::suspend_always{}; }
        void unhandled_exception() { std::terminate(); }
        void return_void() {}
        auto yield_value(int value) {
            current_value = value;
            return std::experimental::suspend_always{};
        }
    };
    bool move_next() { return coro ? (coro.resume(), !coro.done()) : false; }
    int current_value() { return coro.promise().current_value; }
    generator(generator const&) = delete;
    generator(generator && rhs) : coro(rhs.coro) { rhs.coro = nullptr; }
    ~generator() { if (coro) coro.destroy(); }
private:
    generator(handle h) : coro(h) {}
    handle coro;
};

generator f() { co_yield 1; co_yield 2; }

int main() {
    auto g = f();
    while (g.move_next()) std::cout << g.current_value() << std::endl;
}
```

8. Note about possibly undefined behaviour

Section: 8.4.4/11 [dcl.fct.def.coroutine] **Status:** Has wording **Submitter:** US008 **Opened:** 2017-06-05 **Last modified:** 2017-06-05

Proposed change:

Modify note in 8.4.4/11:

If a coroutine has a parameter passed by reference, resuming the coroutine after the lifetime of the entity referred to by that parameter has ended is **likely to result in** undefined behavior.

Proposed resolution:

No change. If after resumption a coroutine does not touch that parameter, there is no undefined behavior.

18. In intro.refs use required text from ISO directive part2

Section: 2 [intro.refs] **Status:** Has wording **Submitter:** CA018 **Opened:** 2017-06-05 **Last modified:** 2017-06-05

Issue:

The form required by ISO/IEC Directives, Part 2, 2016 subclause 15.5.1 is not followed.

Use the text provided by the Directives.

Proposed resolution:

Modify [intro.refs] paragraph 1 as follows:

~~The following referenced document is indispensable for the application of this document. For dated references, only the edition cited applies.~~

The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

21. Wording of 'co_return <expr>;' statement for expressions of type void implies that <expr> is not evaluated

Section: 6.6.3.1 [stmt.return.coroutine] **Status:** Has wording **Submitter:** Lewis Baker **Opened:** 2017-03-11 **Last modified:** 2017-06-19

Issue:

This wording seems to indicate that *expr* is not evaluated in `co_return expr` if the expression has type void, since *expr* does not occur in the translation. I assume this was not the intention here.

Perhaps there needs to be an extra case here to explicitly state what `co_return expr;` translates to if the type of *expr* is void?

Proposed resolution:

Insert new bullet in [stmt.return.coroutine] after 2.2 as follows:

- (2.1) — *S* is *p.return_value*(*braced-init-list*), if the operand is a *braced-init-list*;
- (2.2) — *S* is *p.return_value*(*expression*), if the operand is an expression of non-void type;
- (2.3) — *S* is *expression, return void()*, if the operand is an expression of void type;
- ~~(2.3.4)~~ — *S* is *p.return_void()*, otherwise;

Core comments requesting rebase to C++17

3. Update range based for statement after C++17

Section: 6.5.4/1 [stmt.ranged] **Status:** Comment **Submitter:** US003 **Opened:** 2017-06-05 **Last modified:** 2017-06-05

Comment:

Update range based for statement after C++17

5. Modify co_return grammar to match C++17

Section: 6.6.3.1 [stmt.return.coroutine] **Status:** Comment **Submitter:** US005 **Opened:** 2017-06-05 **Last modified:** 2017-06-05

Comment:

Simplify the grammar for

```
coroutine-return-statement:  
  co_return expression_opt_  
  co_return braced-init-list;
```

to

```
coroutine-return-statement:  
    co_return co_return expr-or-braced-init-list opt;
```

[17. **Rebase entire TS on C++17**](#)

Section: Status: Comment **Submitter:** US017 **Opened:** 2017-06-05 **Last modified:** 2017-06-05

Comment:

We are in the process of balloting the final text of the next C++ standard, provisionally ISO/IEC 14882:2017. We should hold back publishing this TS long enough to rebase on the text of the new standard.

Other than updating this reference, the change is almost entirely updating section numbers and crossreferences. The normative changes would be

- updating the range based 'for' loop syntax;
- the text for a 'return' statement would need adjusting;
- the wording on restrictions with respect to longjmp should be reviewed;
- hash support for coroutine_handle should be updated with the “enabled” terminology.

Core comments (no action)

[4. **It would be good to minimize undefined behaviour**](#)

Section: 6.6.3 6.6.3.1 8.4.4 8.11.2.5 18.10 18.11.2.5 **Status:** Comment **Submitter:** US004 **Opened:** 2017-06-05 **Last modified:** 2017-06-05

Comment:

There are many new cases of undefined behaviour introduced by the TS which are somewhat easily triggered by independent parts of the mechanisms, e.g., the result type of the coroutine interacting through the promise_type to allow flow of control to run off the end of a coroutine. In general it would be good to minimize undefined behaviour.

No action for now. However, experience with TS implementation may allow reducing UB. This should form part of any review for integrating coroutines as part of a future standard.

LWG Issues (with proposed wording)

[9. **Move row in the language support table**](#)

Section: 18.1 [support.general] Table 30 **Status:** Has wording **Submitter:** CA009 **Opened:** 2017-06-05 **Last modified:** 2017-06-05

Issue:

The entry for subclause 18.11 appears before the entry for subclause 18.10.

Proposed resolution:

Move the insertion of the entry for subclause 18.11 to appear after the entry for subclause 18.10.

[10. **Specify the exact behaviour of user-customization of coroutine_traits.**](#)

Section: 18.11.1 [coroutine.traits] **Status:** Has wording **Submitter:** US010 **Opened:** 2017-06-05 **Last modified:** 2017-06-05

Issue:

Is the template `coroutines_traits` intended to be a user-extension point? If so, spell out the contract for users to customize this trait. Otherwise, restrict user specialization with the wording for all type traits in the header. 18.11.1p2 suggests the former, while the latter is much simpler to specify for the initial TS.

Proposed resolution:

1. Remove paragraph 2 of 18.11.1 [`coroutine.traits`]

~~2. Users may specialize `coroutine_traits` to customize the semantics of coroutines.~~

2. In 18.11.1.1 [`coroutine.traits.primary`] add paragraph:

2. A program may specialize this template. Such specialization shall define publicly accessible nested type named `promise_type`.

3. In 18.11.2 [`coroutine.handle`] add paragraph:

2. The behavior of a program that adds specializations for `coroutine_handle` is undefined.

22. Rename [`coroutine.handle.import.export`] to [`coroutine.handle.export.import`] for consistency

Section: 18.11 [`support.coroutine`] **Status:** Has wording **Submitter:** Bryce Lebach **Opened:** 2017-03-10 **Last modified:** 2017-06-19

Issue:

In the class synopsis for `coroutine_handle<>` in [`coroutine.handle`], this section is referred to as "export/import", and the export function (`address`) is listed before the import function (`from_address`). Likewise, in the definitions for these two methods in [`coroutine.handle.import.export`], the section is titled "Export/import" and `address` appears first. Since `from_address` mentions `address`, this seems like the correct order to list things in as it avoids adding forward references to the spec. I'd like to rename this stable tag from [`coroutine.handle.import.export`] to [`coroutine.handle.export.import`] as an editorial change before PTDS.

[Pull request.](#)

Proposed resolution:

`s/coroutine.handle.import.export/coroutine.handle.export.import/g`

23. `coroutine_handle::from_address` - consolidate duplicate definitions, add missing `constexpr` and replace `address()` with `address` in precondition wording

Section: 18.11.2 [`coroutine.handle`] **Status:** Has wording **Submitter:** Bryce Lebach **Opened:** 2017-03-10 **Last modified:** 2017-06-19

Issue:

There are currently two definitions of the `from_address`: one in [`coroutine.handle.import.export`] and one in [`coroutine.handle.import`]. In the class synopses in [`coroutine.handle`], the `coroutine_handle<>` specialization references [`coroutine.handle.import.export`] while primary definition for `coroutine_handle` references [`coroutine.handle.import`]. They are nearly identical in wording, although the definition in [`coroutine.handle.import.export`] is written as if it was out of line (e.g. `coroutine_handle<>::from_address`). Even though the primary template of `coroutine_handle` inherits from `coroutine_handle<>` it is necessary to define `from_address` in the primary template, since, `from_address` returns `coroutine_handle`, which is a different type in the primary template than it is in `coroutine_handle`.

`from_address`'s *Requires*: paragraph in [`coroutine.handle.import.export`] states the pre-condition that "`addr` was obtained via a prior call to `address()`". It should be `address`, not `address()`, since `address()` is an expression not a method

`from_address` is declared `constexpr` in the class synopsis ([`coroutine.handle`], in both the primary template and the specialization for `coroutine_handle<>`) but is not `constexpr` in the definition. The design intent, I believe, is for `from_address` to be `constexpr`.

[Pull request.](#)

Proposed resolution:

1. Modify 18.11.2 [coroutine.handle] primary template synopsis:

```
// 18.11.2.3 import  
// 18.11.2.2 import  
constexpr static coroutine_handle from_address(void* addr);
```

2. Modify 18.11.2.2 [coroutine.handle.import.export] as follows:

```
constexpr static coroutine_handle<> coroutine_handle<>::from_address(void* addr);  
constexpr static coroutine_handle<Promise> coroutine_handle<Promise>::from_address(void* addr);  
Requires: addr was obtained via a prior call to address(r).
```

LWG Issues (no wording)

[11.](#) **coroutine_handle: Unclear where specification refer to specialization or primary template**

Section: 18.1.2 [coroutine.handle] **Status:** Comment **Submitter:** US011 **Opened:** 2017-06-05 **Last modified:** 2017-06-05

Issue:

The specification of each operation is not explicitly clear whether it applied to the specialization of `coroutine_handle<void>`, or the primary `coroutine_handle` template.

Break this section into two, to clearly provide definitions for both versions of the template.

Proposed resolution:

Use resolution for [23](#)

[12.](#) **Should there be a coroutine_handle type with ownership semantic?**

Section: 18.1.2 [coroutine.handle] **Status:** Comment **Submitter:** US012 **Opened:** 2017-06-05 **Last modified:** 2017-06-05

Comment:

Coroutine handles have essentially raw pointer semantics. Should there be a library type as part of the TS that does destroy / set to nullptr?

If a library type is needed, please add it.

Discussion:

coroutine handle is a low level type. Ownership semantic is introduced by higher level types such as a generator or task. Note that not every use of `coroutine_handle` requires ownership semantic. An iterator does not own the coroutine nor a `coroutine_handle` captured by a lambda passed as a callback parameter to an asynchronous API owns the coroutine it refers to.

Proposed resolution:

No action

[14.](#) **Comment about: a concurrent resumption of a coroutine by multiple threads may result in a data race**

Section: 18.11.2.5 **Status:** Comment **Submitter:** US014 **Opened:** 2017-06-05 **Last modified:** 2017-06-05

Comment:

a concurrent resumption of a coroutine by multiple threads may result in a data race

Possibly means concurrent destruction here, in the destroy method.

Discussion:

Yes. Any combination of resumptions may result in a data race: resume/resume, resume/destroy or destroy/destroy.

Proposed resolution:

No action

15. Make coroutine_handle comparison constexpr

Section: 18.11.2.7 **Status:** Comment **Submitter:** US015 **Opened:** 2017-06-05 **Last modified:** 2017-06-05

Comment:

As `coroutine_handle<void>` is a literal type, should the comparison operators be `constexpr`?

Add `constexpr` to the declaration/definition of `operator==`, `operator !=`, `operator<`, `operator<=`, `operator>=`, and `operator>` for arguments of type `coroutine_handle<>`.

Discussion:

The only literal `coroutine_handle` is the default constructed one. Not sure if we need `constexpr` on comparisons.

Proposed resolution:

No action

16. Rename suspend_always to suspend_always_t and suspend_never to suspend_never_t

Section: 18.11.3 **Status:** Comment **Submitter:** US016 **Opened:** 2017-06-05 **Last modified:** 2017-06-05

Comment:

The names `suspend_never` and `suspend_always` should be (inline) `constexpr` variables of type `suspend_never_t` and `suspend_always_t` respectively.

Change `suspend_never` and `suspend_always` as appropriate.

Discussion:

Most common pattern observed in the wild for these types are:

```
struct promise_type {
    suspend_never initial_suspend() { return {}; }
    suspend_always yield_value(int value) { ...; return {}; }
    ...
};
```

Suggested change makes the common case more verbose.

Proposed resolution:

No action

19. Add higher-level coroutine types

Section: **Status:** Comment **Submitter:** US019 **Opened:** 2017-06-05 **Last modified:** 2017-06-05

Comment:

The TS presents only low level mechanisms to implement coroutines. For final release in a C++ standard, standard library implementations of generators, futures from coroutines, guard types for handles, etc. should also ship.

Please consider adding standard library implementations of generators, futures from Coroutines, guard types for handles and any others that may be needed when Coroutines are incorporated into the C++ Standard.

Discussion:

Yes. We plan to add generator and adapters for network TS and concurrency TS.

Proposed resolution:

No immediate action for PDTS

[20. Disallow storing coroutines in std::function objects that discard their result.](#)

Section: Status: Comment **Submitter:** US020 **Opened:** 2017-06-05 **Last modified:** 2017-06-05

Comment:

Coroutines are invokable types, can they be stored by a std::function? What about a std::function<void()> that discards the result on invocation?

Disallow storing coroutines in std::function objects that discard their result.

Discussion:

Yes, coroutines are functions and they can be stored in a std::function. Storing a coroutine in a std::function objects that discard their result is no different than storing a function with the same signature as coroutine in std::function.

Proposed resolution:

No action

Evolution Issues (no wording)

[1. Support stackful coroutines](#)

Section: Status: Comment **Submitter:** CH001 **Opened:** 2017-06-05 **Last modified:** 2017-06-05

Issue:

This TS disallows stackful coroutines. This is too restrictive and stackful coroutines should be allowed as well.

Allow as suspension context functions that were called from a top-level coroutine.

[13. Allow both return_void and return_value](#)

Section: Status: Comment **Submitter:** US013 **Opened:** 2017-06-05 **Last modified:** 2017-06-05

Comment:

Promise types are required to implement either return_value() or return_void(), but not both, and it is undefined behaviour for a coroutine to run off the end, where return_void would be called.

Consider implementing both either_return() and return_value() for promise types, and eliminate the undefined behaviour.

Discussion:

The design intent of having one or the other to make sure that co_return behavior in a coroutine is similar to that of return in a regular function. Allowing both would allow this code to compile:

```
coro f(bool cond) {  
    if (cond)  
        co_return 42;  
    else  
        co_return;  
}
```


which we would like to avoid.

Proposed Resolution:

No change